

Project Status Report

Real-Time Speech Translation System

Manmadha (2025MCS2101)

Jithendra (2025MCS2743)

March 20, 2026

Current Phase Status

Phase	Description	Day	Status
1	Audio Ingress + Dashboard	Day 20	Complete
2	ASR Integration (faster-whisper)	Day 40	Complete
3	Translation Integration (NLLB)	Day 70	In Progress
4	TTS + Full E2E Pipeline	Day 80	Planned

This report covers Phases 1, 2, and the initial integration of Phase 3. The system currently performs live microphone capture, utterance segmentation using a neural Voice Activity Detection model, offline Hindi ASR, offline Hindi-to-Telugu translation, and real-time dashboard observability — all without network connectivity.

1 Project Objective

The objective of this project is to build a real-time speech-to-speech translation system for Indian languages. The intended direction is Hindi to Telugu. The final target is a continuous streaming system in which live spoken Hindi is captured from a microphone, recognized into text, translated into Telugu, synthesized as speech, and played back through a speaker.

The planned end-to-end pipeline is:

Hindi Speech → ASR → Hindi-to-Telugu Translation → TTS → Speaker Output

The project is developed incrementally in phases because the final system must satisfy multiple constraints simultaneously: low latency, continuous operation, modularity, offline execution, and observability. At the current stage, the system performs real-time microphone capture, utterance segmentation, Hindi ASR, Hindi-to-Telugu translation, and live dashboard observability.

2 Current Scope of Implementation

The current implementation corresponds to a working intermediate version of the system. It does not yet produce speech output but validates the real-time backbone that TTS will depend on. The present system performs the following live pipeline:

Microphone → [audio_queue] → Silero-VAD → faster-whisper ASR →
[asr_to_trans_queue] → NLLB-600M → state{} → WebSocket → Dashboard

The application continuously captures microphone audio, identifies spoken utterances using a neural VAD model, runs ASR on those utterances, translates the recognized Hindi text into Telugu offline, and displays all pipeline state in a browser-based dashboard.

3 Why This Phase Matters

This phase is important because ASR and translation are the first two stages that convert raw audio into meaningful translated text. Without working ASR and translation, speech synthesis cannot be meaningfully integrated. This phase also validates the architectural assumptions of the overall project. Specifically, it demonstrates that:

- Live audio can be captured continuously without blocking the application.
- Speech can be segmented into processable utterances inside a streaming loop.
- An offline ASR model can be integrated into the runtime pipeline with controllable latency.
- Offline neural translation can be decoupled from ASR and run in a separate thread.
- Dashboard observability can be maintained while all processing stages run concurrently.

4 Current System Architecture

The long-term target architecture is:

```
Microphone → ASR → Translation → TTS → Speaker
```

The currently implemented architecture is:

```
Microphone → [audio_queue] → Silero-VAD → faster-whisper ASR →  
[asr_to_trans_queue] → NLLB-600M → state{} → WebSocket → Dashboard
```

The architecture is deliberately stage-oriented. Each stage has a well-defined responsibility and communicates through bounded queues rather than tightly coupled logic. This improves both debuggability and extensibility. The same backbone will later allow the TTS stage to be inserted without modifying existing stages.

Stage	Technology	Interface	Status
Audio Capture	sounddevice 16kHz / 32ms mono	audio_queue (max=200)	Live
VAD Segmenter	Silero-VAD confidence > 0.6	Utterance buffer	Live
ASR	faster-whisper small int8/CPU	asr_to_trans (max=50)	Live
Translation	NLLB-200-distilled-600M CPU	state{} WebSocket	Live
Dashboard	FastAPI + WebSocket 10Hz	Browser	Live
TTS (Phase 4)	IndicTTS (planned)	Speaker output	Planned

Table 1: System Pipeline Stages

4.1 Audio Ingress Stage

Microphone audio is captured in callback mode using sounddevice at 16 kHz with 32 ms frames (512 samples). The frame size was increased from 20 ms during development because 20 ms frames (320 samples) caused a runtime crash in Silero-VAD, which requires at least 512 samples per call.

4.2 Buffering and Queue Design

Frames are inserted into a bounded queue (maxsize=200) using `put_nowait` — the callback never blocks. A second bounded queue (`asr_to_trans_queue`, maxsize=50) separates ASR output from the translation stage. Both queues enforce backpressure: if downstream inference is slow, frames are dropped rather than accumulating unboundedly in memory.

4.3 Voice Activity Detection Stage

The original implementation used a simple energy threshold (`START_THRESHOLD = 0.02`) which triggered on fan noise, keyboard sounds, and background audio, causing ASR to process garbage frames. It required manual tuning per environment.

This was replaced with Silero-VAD, an offline neural model that returns a speech confidence score per 32 ms frame. A threshold of 0.6 was found to eliminate false positives without missing soft speech.

Attribute	Energy Threshold (Before)	Silero-VAD (After)
Detection method	RMS energy > 0.02	Neural classifier, confidence > 0.6
Noise rejection	None — triggers on any sound	Trained on real speech vs. noise
False positives	High — fan, keyboard, ambient	Near-zero in all testing
Portability	Requires manual tuning per env	Fixed threshold, portable
Latency per frame	< 1ms	~2ms on CPU

Table 2: VAD Comparison

4.4 ASR Stage

The recognition stage uses faster-whisper with the small model and int8 quantization, providing approximately 3x faster inference compared to openai-whisper on CPU. Hindi (hi) is hard-coded as the source language to avoid detection overhead.

During testing, two classes of hallucination were observed when near-silence frames reached Whisper:

```
Output: "██ ██ ███ ███..." (5901ms consumed) Output: "██████████████████..." (5690ms consumed)
```

Three parameters were applied together to eliminate these outputs entirely:

Parameter	Value	Effect
no_speech_threshold	0.6	Discard if Whisper's own speech confidence < 60%
compression_ratio_threshold	2.4	Discard repetitive output — repeated tokens compress highly
log_prob_threshold	-1.0	Discard very low probability token sequences

Table 3: Hallucination Suppression Parameters

After applying these parameters, zero hallucinations were observed. Typical ASR latency fell from sporadic 5900ms spikes to a consistent 1300–1600ms per utterance.

4.5 Translation Stage

Phase 3 adds an offline Hindi-to-Telugu translation thread that consumes from `asr_to_trans_queue` and writes results to shared state. This decoupling ensures that translation inference does not block ASR or audio capture.

Several models were evaluated:

Model	Size	Hindi→Telugu	Offline	Decision
IndicTrans2-indic-indic-1B	~4 GB	Best	Yes	Too large
IndicTrans2-dist-200M	~800 MB	Good	Yes	Not on HuggingFace hub
Helsinki-NLP opus-mt-te-hi	~300 MB	Wrong dir.	Yes	Incorrect direction
NLLB-200-distilled-600M	~600 MB	Good	Yes	Selected
googletrans	API	Good	No	Requires internet

Table 4: Translation Model Comparison

NLLB-200-distilled-600M was selected. The tokenizer requires `tok.src_lang = 'hin_Deva'` before tokenization, and the target language is enforced via `forced_bos_token_id = tok.convert_tokens_to_ids('tel_Telu')`.

4.6 Dashboard and Backend Communication

The dashboard is served by FastAPI and connected via WebSocket. State is broadcast at 10Hz. Auto-reconnect logic was added — the browser retries every 2 seconds without requiring a page reload. The dashboard currently displays: live audio energy, frame count, runtime, liveness indicators, speech/silence status, ASR latency, recognized Hindi text, translated Telugu text, translation latency, and end-to-end latency from speech onset to translation complete.

5 Detailed Current Functionality

The system performs the following real-time tasks without manual triggering:

- Continuous microphone capture without offline file dependency.
- Each captured frame is processed online through a queue-based pipeline.
- Silero-VAD runs per 32ms frame — only speech frames contribute to utterance buffering.
- Valid utterances are transcribed by faster-whisper, producing Hindi text.
- Hindi text is pushed to the translation queue; NLLB-600M translates to Telugu offline.
- All activity — both language outputs, per-stage latency, liveness — reflects live on the dashboard.

6 Technologies and Libraries

Technology	Role
Python	Entire implementation — audio, inference, backend, UI in one codebase
sounddevice	Callback-based live microphone capture at 16kHz
Silero-VAD	Offline neural speech/noise classifier — replaces energy threshold
faster-whisper	Offline Hindi ASR with int8 quantization — 3x faster than reference
NLLB-200-600M	Offline Hindi-to-Telugu translation — 600M parameter multilingual model

Technology	Role
Bounded Queue	Decouples audio capture from ASR and ASR from translation
FastAPI	Lightweight backend serving dashboard and WebSocket endpoint
WebSockets	Push-based 10Hz state broadcast — no polling overhead
HTML/CSS/JavaScript	Browser dashboard with live dynamic updates and auto-reconnect

Table 5: Technologies Used

7 Current Implementation Status

The following have been implemented and tested:

- Streaming audio capture — continuous microphone input without offline file dependency.
- Bounded queue pipeline — audio and processing decoupled through queues.
- Silero-VAD segmentation — neural speech/silence detection per 32ms frame.
- Offline Hindi ASR — faster-whisper small with int8 quantization.
- Hallucination suppression — three-parameter fix, zero repeated-token outputs.
- Offline translation — NLLB-200-distilled-600M Hindi-to-Telugu, fully local.
- Live dashboard — Hindi and Telugu outputs, per-stage latency, liveness indicators.
- End-to-end latency — measured from VAD speech onset to translation complete.
- Offline runtime — no API calls, no network required after model download.

The following are still under development:

- Telugu text-to-speech synthesis (IndicTTS — Phase 4).
- Speaker audio output queue and playback.
- Complete microphone-to-speaker pipeline integration.
- End-to-end latency optimization to below 2000ms.

8 Problems Encountered and Solutions

Problem	Root Cause	Fix Applied
Silero-VAD crash: chunk too short	20ms frame = 320 samples (< min)	FRAME_DURATION → 0.032s (512 samples)
Hallucination: ■■ ■■ / ■■■■■■■■	Whisper on near-silence frames	compression_ratio=2.4, no_speech=0.6
5900ms ASR latency spikes	Max-duration flush on noise frames	Silero-VAD blocks noise from ASR
IndicTrans2 not found	Not on public HuggingFace hub	Switched to NLLB-200-distilled-600M
Dashboard RuntimeError on start	main.py in src/, dashboard in root	os.path.dirname called twice

Problem	Root Cause	Fix Applied
Telugu text invisible on dashboard	Dark text on dark CSS background	color: var(--text-main) added to CSS
NLLB wrong output language	src_lang not set before tokenize	tok.src_lang='hin_Deva' before tok()
Translation error: lang_code_to_id	TokenizersBackend attribute missing	tok.convert_tokens_to_ids('tel_Telu')
WebSocket drops on server restart	No reconnect logic in script.js	setTimeout(connect, 2000) added

Table 6: Problems, Root Causes, and Fixes Applied

9 Performance Observations

Metric	Observed Value	Target
Audio frame size	32ms / 512 samples	≤ 32ms (Silero minimum)
Dashboard update rate	10 Hz	10 Hz
ASR latency — typical	1300–1600ms	< 2000ms
ASR latency — before fix	Up to 5900ms (spikes)	Eliminated
VAD overhead per frame	~2ms on CPU	< 5ms
Hallucinations	Zero after fix	Zero
E2E latency (ASR + Trans)	~3000ms currently	< 2000ms (Phase 4)
RTF — ASR stage only	~0.72	< 1.0
Continuous runtime	Stable — 10+ min	Stable

Table 7: Performance Summary

The Real-Time Factor (RTF) for the ASR stage is approximately 0.72, confirming real-time operation. $RTF = \text{processing_time} / \text{audio_duration}$; a value below 1.0 means the system processes audio faster than it arrives. Combined with translation latency, the current end-to-end RTF is slightly above 1.0 — this will be addressed through pipeline overlap in Phase 4.

10 GitHub Repository

The project source code is maintained at:

github.com/ManmadhaKuttum/CornerStone_COP7001

11 GitHub Change Log

Commit	Message
bfd914b	segmenter: replace energy threshold VAD with Silero-VAD neural model
2734c11	dashboard: fix speech_active field, add auto-reconnect, color-coded latency

Commit	Message
0eeb998	dashboard: remove speaker output panel, clean 3-row layout fits single screen
9f50337	dashboard: fix finalized text color visibility, remove speaker panel
8a4f8f3	main: fix dashboard path, start translation thread, extend WebSocket payload
271573d	translation: add NLLB-200-distilled-600M offline Hindi-to-Telugu thread
4cad7da	asr: add hallucination suppression parameters to faster-whisper
ee2d003	audio: increase frame size to 32ms for Silero-VAD compatibility
2182291	state: add translation and E2E latency fields to shared state
8b193de	config: add MPS/CUDA device detection, VAD and translation parameters
2a72136	configuration changes device and compute type settings
a416e9e	Phase 2: Added Hindi ASR using faster-whisper with real-time dashboard updates
84d4aa9	adding phase-1 technical design document
53b1e14	Clean requirements.txt for Phase 1 (cross-platform)
05be995	Phase 1: Replace legacy stream module with audio ingress pipeline and dashboard
22325b9	Initial commit

Table 8: Git Commit History

12 Next Steps

The next phase will complete the microphone-to-speaker pipeline:

1. Integrate IndicTTS (AI4Bharat) for offline Telugu speech synthesis.
2. Implement a TTS output queue between translation and speaker playback.
3. Measure TTS latency and include it in the dashboard end-to-end display.
4. Optimize the pipeline for end-to-end latency below 2000ms through stage overlap.
5. Conduct a complete offline demonstration: speak Hindi, hear Telugu.

13 Conclusion

The current submission represents a technically meaningful intermediate stage. The implemented system demonstrates all real-time backbone components required by the project specification: streaming audio capture, neural voice activity detection, utterance segmentation, offline Hindi ASR with hallucination suppression, offline Hindi-to-Telugu translation, and live dashboard observability — all without network connectivity.

Key engineering contributions include the replacement of naive energy thresholding with Silero-VAD, the identification and elimination of Whisper hallucinations through three-parameter suppression, and the integration of a decoupled offline translation thread with end-to-end latency tracking. These give the project a strong foundation for Phase 4: TTS integration and the final microphone-to-speaker demonstration.